

BABYGUARD

Sistema de Monitoramento Ambiental para Bebês

Programação para Dispositivos Móveis (PPDM)

Robótica

Arthur Lourenço Fritz

Arthur Steiner Morais Silva

Joyce Masalla Jorge

Sânio Rodrigues Silva Trindade

DOCUMENTAÇÃO ACADÊMICA

1. SUMÁRIO

1. Introdução	19. Recursos Nativos
2. Objetivos	20. Comunicação com Hardware
3. Problema Resolvido	21. Integração com Blynk
4. Escopo do Projeto	22. Fluxo de Sincronização
5. Arquitetura Geral	23. Fluxo Offline
6. Tecnologias Utilizadas	24. Telas do Aplicativo
7. Bibliotecas e Dependências	25. Funcionalidades de Cada Tela
8. Estrutura de Pastas	26. Fluxos do Usuário
9. Arquitetura Mobile	27. Endpoints da API
10. Arquitetura Backend	28. Regras de Negócio
11. Banco de Dados PostgreSQL	29. Segurança Implementada
12. Banco de Dados SQLite	30. Tratamento de Erros
13. Estrutura Completa das Tabelas	31. Boas Práticas Aplicadas
14. Autenticação	32. Processo de Instalação
15. Controle de Sessão	33. Processo de Execução
16. Context API	34. Ambiente de Desenvolvimento
17. Gerenciamento de Estado	35. Limitações Conhecidas
18. Navegação	36. Conclusão

1. Introdução

O projeto BabyGuard consiste em um sistema de monitoramento para berços infantis integrado a sensores físicos de temperatura, umidade, luminosidade e distância (ultrassônico) para rastrear movimento. O sistema é composto por um hardware embarcado (ESP32 simulado via Wokwi), um serviço de nuvem intermediário (Blynk IoT Cloud), uma API REST de backend (Node.js/Express com banco PostgreSQL) e um aplicativo móvel (React Native/Expo com banco local SQLite e AsyncStorage).

2 Objetivos

- Mapear e persistir localmente as leituras ambientais e alertas do berço.
- Disponibilizar as informações em tempo real no aplicativo móvel.
- Garantir o funcionamento contínuo do aplicativo móvel de maneira offline através de cache local estruturado no banco SQLite.
- Fornecer controle de acesso autenticado (JWT) para os usuários no backend e no aplicativo.
- Oferecer uma rotina de standby (modo economia/pausa) controlada diretamente pelo aplicativo via comunicação com o hardware.

3. Problema Resolvido

A falta de acompanhamento em tempo real das condições de conforto térmico, luminosidade excessiva e atividade física (movimentação ou ausência do bebê no berço), bem como a instabilidade de rede que comumente inutiliza aplicativos puramente online. O BabyGuard soluciona essa interrupção aplicando uma arquitetura offline-first no aplicativo, permitindo a consulta do histórico mesmo sem conexão ativa à internet.

4. Escopo do Projeto

O escopo abrange o pareamento e renomeação de dispositivos de monitoramento (robôs), a visualização dos dados dos sensores em formato de cartões visuais com barras de progresso, um gráfico histórico das variações de temperatura local, o recebimento de alertas dinâmicos baseados em limites configuráveis do aplicativo, a edição de dados de perfil do usuário e a alternância global de tema claro/escuro.

5. Arquitetura Geral

O ecossistema é distribuído em quatro camadas:

37. **Embarcado (ESP32)**: Coleta dados físicos (Temperatura/Umididade com DHT11, Luminosidade com LDR, Distância com HC-SR04) e transmite ao Blynk via protocolo TCP/IP.

38. **Blynk Cloud**: Recebe e mantém em cache os valores dos pinos virtuais.

39. **Backend API:** Um servidor Node.js/TypeScript que consulta periodicamente (de 10 em 10 segundos) a API REST do Blynk, insere os dados no banco PostgreSQL e fornece endpoints REST para o aplicativo.
40. **Mobile App:** Consome a API do backend, gerencia sessões locais via AsyncStorage e armazena os dados localmente no SQLite.

6. Tecnologias Utilizadas

- **Mobile:** React Native, Expo SDK 54, TypeScript, React Navigation, React Native Paper.
- **Backend:** Node.js, Express, TypeScript, ts-node-dev, pg (PostgreSQL driver).
- **Bancos de Dados:** PostgreSQL 16 (nuvem/produção) e SQLite 3 via `expo-sqlite` (banco local embarcado).
- **IoT/Hardware:** ESP32, sensores DHT11, HC-SR04, LDR, Blynk Cloud API.

7. Bibliotecas e Dependências

2.7.1. Frontend (`frontend/package.json`)

- `@expo/vector-icons` (v15.0.3)`: Renderização e gerenciamento de ícones vetoriais nativos (Ionicons) em abas de navegação, cabeçalhos e elementos interativos do aplicativo.
- `@react-native-async-storage/async-storage` (v2.2.0)`: Armazenamento local leve chave-valor para manter o estado da sessão (token JWT, dados do usuário) e preferências do aplicativo (Tema Claro/Escuro).
- `expo-sqlite` (v16.0.10)`: Driver assíncrono do banco de dados local SQLite 3, essencial para persistência estruturada do cache offline de leituras, alertas, limites de sensores e dispositivos.
- `expo-image-picker` (v17.0.11)`: Componente que solicita permissões e gerencia o uso da Câmera nativa e da Galeria de Fotos do celular para atualização da imagem de perfil.
- `@react-navigation/native` (v7.2.4)` e `@react-navigation/bottom-tabs` (v7.16.0)`: Infraestrutura principal de roteamento do aplicativo móvel, provendo a navegação inferior de abas principais.

- **`@react-navigation/native-stack` (v7.15.0):** Roteador de pilha (Stack) para gerenciar fluxos de telas secundárias (Login, Register, NovoRobo, RenomearRobo, RoboDetalhes, Perfil).
- **`react-native-chart-kit` (v6.12.3):** Biblioteca para plotar o gráfico de variações históricas de temperatura na Dashboard.
- **`react-native-svg` (v15.12.1):** Renderizador de gráficos vetoriais necessário como dependência do `react-native-chart-kit`.
- **`react-native-paper` (v5.15.3):** Biblioteca de componentes UI baseada em Material Design (botões, inputs de formulário, banners e switchers de tema).
- **`react-native-toast-message` (v2.3.3):** Exibição de alertas visuais rápidos (Toasts) flutuantes na tela para feedbacks de ações do usuário.
- **`react-native-safe-area-context` (v5.6.0) e `react-native-screens` (v4.16.0):** Bibliotecas de otimização de telas nativas e posicionamento seguro contra recortes de telas (Notch).
- **`react-dom` (v19.1.0) e `react-native-web` (v0.21.0):** Dependências mantidas no projeto para garantir compatibilidade de compilação e suporte ao ambiente Web do Expo.
- **`expo-asset` (v12.0.13):** Biblioteca para gerenciar e pré-carregar recursos de mídia locais na inicialização.

2.7.2. DevDependencies Frontend

- **`jest` (v30.4.2) e `jest-expo` (v56.0.4):** Framework de execução de testes unitários adaptado para o ecossistema Expo.
- **`@testing-library/react-native` (v14.0.0):** Utilitários para renderização e teste de comportamento de componentes React Native.
- **`@types/jest` (v29.5.0), `@types/react` (v19.1.0) e `typescript` (v5.9.2):** Ferramentas de tipagem estática e compilação do React Native.

2.7.3. Backend (`backend/package.json`)

- **`express` (v4.21.2):** Framework web HTTP para construção de APIs REST e gerenciamento de rotas e middlewares.

- **`pg` (v8.13.1):** Driver cliente PostgreSQL para execução de consultas parametrizadas diretas ao banco Supabase.
- **`jsonwebtoken` (v9.0.3):** Emissão e validação de chaves de assinatura do protocolo de segurança HMAC-SHA256 (Tokens JWT) no backend.
- **`bcryptjs` (v3.0.3):** Criptografia e hashing seguro de senhas no registro e validação do login do usuário.
- **`dotenv` (v16.6.1):** Inicialização das configurações de desenvolvimento e carregamento de variáveis de ambiente do arquivo `.env`.
- **`cors` (v2.8.5):** Habilitação de requisições de origem cruzada para conexões seguras originadas do aplicativo móvel.
- **`axios` (v1.16.1):** Cliente HTTP assíncrono consumido pela rotina do worker (`blynkSyncService.ts`) para sincronização automática com a API do Blynk IoT Cloud.
- **`ts-node` (v10.9.2) e `ts-node-dev` (v2.0.0):** Executam scripts TypeScript diretamente no ambiente de desenvolvimento com reload automático.
- **`@types/bcryptjs`, `@types/cors`, `@types/express`,
`@types/jsonwebtoken`, `@types/node`, `@types/pg`, `@types/uuid`:** Pacotes de tipagem estática TypeScript para segurança em tempo de compilação.

8. Estrutura de Pastas

```
BabyGuard
├── backend
│   ├── dist (código compilado)
│   └── src
│       ├── config (database.ts)
│       ├── controllers (auth, blynk, dispositivos, eventos, leituras,
│       │               sensores, usuarios)
│       ├── middleware (autenticacao, tokenBlacklist)
│       ├── models (interfaces TypeScript)
│       ├── routes (definições de rotas HTTP)
│       ├── services (regras de negócio e queries ao banco pg/Blynk)
│       ├── types (index.ts)
│       ├── utils (respostas, validacao)
│       ├── app.ts (configuração express)
│       └── server.ts (entrada do servidor)
├── package.json
└── frontend
    ├── src
    │   ├── config (apiUrl.ts)
    │   ├── context (AuthContext, ThemeContext)
    │   ├── controllers (roboController.ts)
    │   ├── database (sqlite.ts, migrations/schema.ts)
    │   ├── hooks (useLogin, usePerfil)
    │   ├── models (interfaces locais)
    │   ├── repositories (Dispositivo, Evento, Leitura, Sensor, SensorLimits)
    │   ├── routes (RootNavigator, TabNavigator)
    │   ├── services (auth, blynk, dispositivos, eventos, leituras)
    │   ├── shared (styles, utils/logger)
    │   ├── styles (folhas de estilo CSS-in-JS)
    │   ├── tests (Jest tests)
    │   └── views (components, screens)
    └── package.json
```

9. Arquitetura Mobile

O frontend adota o padrão **MVVM / Repository**:

- **Views:** Telas React Native que renderizam a interface.
- **Controllers:** Intermediam a lógica entre o repositório local e a interface (ex: [roboController.ts](../frontend/src/controllers/roboController.ts)).
- **Services:** Tratam requisições HTTP externas e injetam fallbacks para o banco local.
- **Repositories:** Executam queries diretas no banco SQLite.
- **Models:** Definem as tipagens de dados das tabelas locais.

10. Arquitetura Backend

O backend segue o modelo **Controller-Service-Repository (Database Access)**:

- **Routes**: Direcionam chamadas HTTP externas aos controllers.
- **Middlewares**: Interceptam requisições para controle de sessão (autenticação JWT) e blacklist de tokens de logout.
- **Controllers**: Validam o payload JSON de entrada (`validacao.ts`) e invocam a camada de serviços.
- **Services**: Concentram a lógica de negócios e efetuam queries brutas parametrizadas utilizando o pool do PostgreSQL (`pg`).

11. Banco de Dados PostgreSQL

O banco PostgreSQL hospeda centralizadamente todos os dados dos usuários e dispositivos. Ele é hospedado na nuvem (Supabase) e acessado pelo backend usando as credenciais definidas na variável de ambiente `DATABASE_URL` no arquivo `.env`.

12. Banco de Dados SQLite

O banco de dados SQLite (`babyguard.db`) roda localmente no dispositivo móvel do usuário por meio do driver assíncrono `expo-sqlite`. Ele é inicializado no arquivo `[sqlite.ts](../frontend/src/database/sqlite.ts)` e configurado na inicialização do aplicativo em `[App.tsx](../frontend/src/App.tsx)`.

13. Estrutura Completa das Tabelas

2.13.1. PostgreSQL (Produção/Nuvem)

41. usuarios

- `id_usuario` (SERIAL, PRIMARY KEY)

- nome (VARCHAR)
- email (VARCHAR, UNIQUE)
- senha_hash (VARCHAR)
- telefone (VARCHAR, NULLABLE)
- foto_perfil (TEXT, NULLABLE)
- push_token (VARCHAR, NULLABLE)
- status_usuario (VARCHAR, DEFAULT 'ativo')
- criado_em (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

42. dispositivos

- id_dispositivo (SERIAL, PRIMARY KEY)
- uuid_dispositivo (VARCHAR, UNIQUE)
- id_usuario (INTEGER, FOREIGN KEY REFERENCES usuarios)
- nome_dispositivo (VARCHAR)
- wifi_ssid (VARCHAR, NULLABLE)
- token_dispositivo (VARCHAR, NULLABLE)
- status_dispositivo (VARCHAR, DEFAULT 'offline')
- movimento (BOOLEAN, DEFAULT FALSE)
- ativo (BOOLEAN, DEFAULT TRUE)
- criado_em (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

43. sensores

- id_sensor (SERIAL, PRIMARY KEY)
- id_dispositivo (INTEGER, FOREIGN KEY REFERENCES dispositivos)
- nome_sensor (VARCHAR)
- tipo_sensor (VARCHAR)
- unidade_medida (VARCHAR, NULLABLE)
- descricao (VARCHAR, NULLABLE)

44. leituras

- id_leitura (SERIAL, PRIMARY KEY)
- id_sensor (INTEGER, FOREIGN KEY REFERENCES sensores)

- `valor` (NUMERIC, NULLABLE)
- `valor_booleano` (BOOLEAN, NULLABLE)
- `movimento` (BOOLEAN, NULLABLE)
- `data_hora` (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

45. eventos

- `id_evento` (SERIAL, PRIMARY KEY)
- `id_dispositivo` (INTEGER, FOREIGN KEY REFERENCES dispositivos)
- `id_sensor` (INTEGER, FOREIGN KEY REFERENCES sensores, NULLABLE)
- `tipo_evento` (VARCHAR)
- `nivel_criticidade` (VARCHAR)
- `data_evento` (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

46. tokens_revogados

- `token` (TEXT, PRIMARY KEY)
- `revogado_em` (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

2.13.2. SQLite (Embarcado/Local)

As definições constam no arquivo

[schema.ts](../frontend/src/database/migrations/schema.ts):

47. dispositivos

- `id_dispositivo` (INTEGER, PRIMARY KEY AUTOINCREMENT)
- `uuid_dispositivo` (TEXT, UNIQUE, NOT NULL)
- `id_usuario` (INTEGER)
- `nome_dispositivo` (TEXT, NOT NULL)
- `status_dispositivo` (TEXT, DEFAULT 'offline')
- `ativo` (INTEGER, DEFAULT 1)
- `criado_em` (DATETIME, DEFAULT CURRENT_TIMESTAMP)
- `atualizado_em` (DATETIME, DEFAULT CURRENT_TIMESTAMP)

48. sensores

- `id_sensor` (INTEGER, PRIMARY KEY AUTOINCREMENT)
- `id_dispositivo` (INTEGER, NOT NULL, FOREIGN KEY REFERENCES dispositivos)
- `nome_sensor` (TEXT, NOT NULL)
- `tipo_sensor` (TEXT, NOT NULL)
- `unidade_medida` (TEXT)
- `criado_em` (DATETIME, DEFAULT CURRENT_TIMESTAMP)

49. **limites_sensores**

- `id_limite` (INTEGER, PRIMARY KEY AUTOINCREMENT)
- `label` (TEXT, UNIQUE, NOT NULL)
- `tipo_sensor` (TEXT, NOT NULL)
- `min` (REAL, NOT NULL)
- `max` (REAL, NOT NULL)
- `unidade_medida` (TEXT)
- `atualizado_em` (DATETIME, DEFAULT CURRENT_TIMESTAMP)

50. **leituras**

- `id_leitura` (INTEGER, PRIMARY KEY AUTOINCREMENT)
- `id_sensor` (INTEGER, NOT NULL, FOREIGN KEY REFERENCES sensores)
- `valor` (REAL)
- `valor_booleano` (INTEGER)
- `movimento` (INTEGER)
- `data_hora` (DATETIME, DEFAULT CURRENT_TIMESTAMP)
- `sincronizado` (INTEGER, DEFAULT 0)

51. **eventos**

- `id_evento` (INTEGER, PRIMARY KEY AUTOINCREMENT)
- `id_dispositivo` (INTEGER, NOT NULL, FOREIGN KEY REFERENCES dispositivos)
- `id_sensor` (INTEGER, FOREIGN KEY REFERENCES sensores, NULLABLE)
- `tipo_evento` (TEXT, NOT NULL)

- `nivel_criticidade` (TEXT, DEFAULT 'normal')
- `data_evento` (DATETIME, DEFAULT CURRENT_TIMESTAMP)
- `sincronizado` (INTEGER, DEFAULT 0)

2.13.3. Modelo Relacional Textual

```
[usuarios] 1 ----- 0..N [dispositivos]
[dispositivos] 1 -- 0..N [sensores]
[dispositivos] 1 -- 0..N [eventos]
[sensores] 1 ----- 0..N [leituras]
[sensores] 0..1 --- 0..N [eventos]
```

14. Autenticação

A autenticação do sistema é realizada via token JWT (JSON Web Token).

- O backend gera o token na rota `/auth/login` ou `/auth/registro` contendo o `id_usuario` e o `email` com expiração padrão.
- O token é transmitido no cabeçalho HTTP da requisição: `Authorization: Bearer <token>`.
- O middleware `autenticacao.ts` no backend intercepta as rotas protegidas e valida a assinatura do token.
- Ao deslogar (POST `/auth/logout`), o token é armazenado na tabela PostgreSQL `tokens_revogados` (blacklist) para impedir sua reutilização posterior antes de expirar.

15. Controle de Sessão

No aplicativo móvel, a sessão do usuário é gerenciada localmente por meio do `AsyncStorage`.

- Ao logar/registrar, as funções persistirão o token sob a chave `'token'` e o objeto do usuário na chave `'usuario'`.
- Ao iniciar o aplicativo, o `AuthContext` lê estas chaves para determinar se direciona o fluxo para a tela de Login ou de Dashboard.
- Ao clicar em "Sair da Conta", as chaves `'token'` e `'usuario'` são excluídas do `AsyncStorage` localmente e o token é enviado para revogação no backend.

16. Context API

O aplicativo móvel implementa dois contextos globais:

2.16.1. AuthContext

- **Estado:** `token` (string | null), `usuario` (DadosUsuario | null), `estaLogado` (boolean), `carregandoSessao` (boolean).
- **Métodos:**
 - `salvarSessao(token, usuario)`: Persiste dados do login/cadastro no `AsyncStorage` e atualiza estados.
 - `limparSessao()`: Remove chaves do `AsyncStorage` e limpa estados.
 - `atualizarUsuario(usuario)`: Atualiza os dados de perfil no contexto e no storage.
- **Telas Consumidoras:** `RootNavigator`, `LoginScreen`, `RegisterScreen`, `PerfilScreen`, `ConfiguracoesScreen`, `DashboardScreen`.

2.16.2. ThemeContext

- **Estado:** `isDarkMode` (boolean).
- **Métodos:**
 - `toggleDarkMode(value)`: Altera o estado do tema e persiste a preferência no `AsyncStorage` com a chave `@babyguard:darkMode`.
- **Telas Consumidoras:** `AppContent` (`App.tsx`), `TabNavigator`, e todas as 10 telas e componentes estilizados para detecção de cores dinâmicas.

17. Gerenciamento de Estado

Para a tela principal de monitoramento

(`[DashboardScreen.tsx](../frontend/src/views/screens/DashboardScreen.tsx)`), o aplicativo implementa o hook `useReducer` do React para orquestrar estados complexos sob a arquitetura Flux:

- **Estado:**
- `dispositivos`: Lista de robôs pertencentes ao usuário.
- `dispositivoAtivo`: Robô selecionado para monitoramento atual.
- `leituras`: Valores atuais dos sensores do robô selecionado.
- `historicoLeituras`: Histórico das últimas 50 leituras agrupadas para o gráfico.
- `loading`: Flag indicadora de requisições pendentes.
- `isDeviceOn`: Flag que identifica se o dispositivo físico está ligado (modo ativo) ou desligado (standby).
- **Ações:** `START_LOADING`, `SET_DISPOSITIVOS`, `SELECT_DISPOSITIVO`, `SET_LEITURAS`, `SET_DEVICE_STATUS`, `STOP_LOADING`.

18. Navegação

A navegação é estruturada utilizando a biblioteca React Navigation com navegação híbrida Bottom Tabs e Native Stack:

```
Navegação do Aplicativo (RootNavigator Stack)
├── Login (LoginScreen) [Rota Pública / Inicial se não logado]
├── Register (RegisterScreen) [Rota Pública]
├── Tabs (TabNavigator BottomTabs) [Rota Privada / Inicial se logado]
│   ├── Início (DashboardScreen)
│   ├── Alertas (AlertasScreen)
│   ├── Robôs (MeusRobosScreen)
│   └── Ajustes (ConfiguracoesScreen)
├── NovoRobo (NovoRoboScreen) [Rota Privada]
├── RenomearRobo (RenomearRoboScreen) [Rota Privada, Params: id, nome]
├── RoboDetalhes (RoboDetalhesScreen) [Rota Privada, Params: id, nome, local?]
└── Perfil (PerfilScreen) [Rota Privada]
```

19. Recursos Nativos

O aplicativo móvel consome os recursos nativos do celular através do Expo SDK:

52. Câmera: Acionada via `ImagePicker.launchCameraAsync` em `PerfilScreen.tsx` para capturar fotos em tempo real.

53. Acesso à Galeria: Acionado via `ImagePicker.launchImageLibraryAsync` em `PerfilScreen.tsx` para selecionar imagens salvas.

54. **Armazenamento Local:** Acesso físico para persistência local pelo SQLite e AsyncStorage.

55. **Permissões de Câmera/Galeria:** Solicitadas explicitamente pelos métodos

`ImagePicker.requestCameraPermissionsAsync()` e

`ImagePicker.requestMediaLibraryPermissionsAsync()` antes do uso dos recursos nativos.

20. Comunicação com Hardware

A comunicação com o sistema embarcado (ESP32) é indireta e atravessa a internet pública.

56. O ESP32 realiza medições locais físicas e envia os valores para o servidor do Blynk Cloud via protocolo proprietário TCP do Blynk.

57. O backend da API interage com o Blynk Cloud por meio de requisições HTTP REST (Axios) para ler e escrever dados nos pinos virtuais.

58. O aplicativo móvel faz requisições HTTP REST (fetch) para a API do backend para recuperar o histórico e envia chamadas de controle (como standby) diretamente para a API pública do Blynk Cloud.

21. Integração com Blynk

A integração com a plataforma Blynk IoT Cloud é configurada pelos seguintes pinos virtuais:

- **V3:** Temperatura (graus Celsius).
- **V4:** Umidade (porcentagem).
- **V5:** Luminosidade (Lux).
- **V6:** Distância do sensor ultrassônico (centímetros)- mapeada no DB local e remoto como flag bool de movimentação quando a leitura for menor que 50,0 cm.
- **V7:** Estado de Standby do dispositivo ("1" = Dispositivo monitorando/ativo; "0" = Standby/pausado). Quando o Standby é acionado, a placa física ESP32 desliga o LED RGB e escreve -99.0 nos pinos V3, V4, V5 e V6 para alertar o aplicativo
- **V0, V1, V2:** Controle manual dos canais Vermelho, Azul e Verde do LED RGB físico.

22. Fluxo de Sincronização

O sincronismo de dados é orquestrado em duas frentes:

59. Ciclo Backend <-> Blynk: O arquivo

[blynkSyncService.ts](../backend/src/services/blynkSyncService.ts) possui um loop automático (`setTimeout` recursivo) que roda a cada 10 segundos buscando os dados no Blynk. Se o hardware estiver online e o pino V7 for "1", ele grava as leituras no banco PostgreSQL para manter o histórico centralizado.

60. Ciclo Mobile <-> Backend: O aplicativo móvel faz polling periódico a cada 10 segundos enviando requisições REST para a API do backend para recuperar o histórico. Em paralelo, faz um polling mais rápido a cada 5 segundos diretamente na API pública do Blynk no pino V7 para detectar ativação/standby instantaneamente. Ao receber as leituras e alertas dinâmicos da API, o aplicativo os persiste localmente nas tabelas correspondentes do SQLite local para construir o cache offline.

23. Fluxo Offline

Quando o aplicativo móvel perde a conexão com a internet:

61. As funções de busca HTTP em

[leiturasService.ts](../frontend/src/services/leiturasService.ts) falham devido a erro de rede.

62. A cláusula `catch` captura a exceção de rede e redireciona o fluxo para o fallback offline.

63. O fallback offline executa uma query direta de SELECT no SQLite local do aparelho móvel, filtrando o histórico local cadastrado anteriormente.

64. Os dados são convertidos e renderizados na interface gráfica, permitindo que a Dashboard e a lista de Alertas continuem operacionais e legíveis de forma offline.

24. Telas do Aplicativo & 25. Funcionalidades de Cada Tela

2.24.1. Capturas de Tela Reais do Aplicativo (Screenshots)

Para ilustrar a interface gráfica (UI) real e a experiência de usuário (UX) do aplicativo BabyGuard em execução física, seguem as capturas de tela das principais telas do app:

65. Dashboard de Monitoramento (Aba Início)

Exibe as leituras físicas em tempo real recebidas do hardware (temperatura, umidade, luminosidade e presença) com banners dinâmicos de avisos ("Umidade baixa detectada").



Figura 1: Tela Real do Dashboard de Monitoramento

66. Histórico de Leituras Ambientais (Aba Alertas - Leituras)

Exibe a listagem do cache local SQLite com as leituras ambientais minuto a minuto capturadas pelo sistema.



Figura 2: Tela Real do Histórico de Leituras do SQLite

67. Histórico de Alertas de Ocorrências (Aba Alertas - Alertas)

Exibe a listagem de desvios e alarmes calculados pelo app (temperatura baixa, umidade baixa, movimento detectado) com ícones e status de criticidade.



Figura 3: Tela Real do Histórico de Ocorrências do SQLite

68. Gerenciamento de Dispositivos (Aba Robôs)

Exibe os berços inteligentes pareados à conta, com botões para renomear, buscar detalhes ou excluir, além de status dinâmico Online/Offline e Claim Device ("Configurar agora").



Figura 4: Tela Real do Gerenciamento de Robôs

69. Preferências e Ajustes (Aba Ajustes)

Exibe as opções de habilitar alertas, sons, ativar Tema Escuro global do aplicativo e ajustar limites de sensores ideais (temperatura e umidade).



Figura 5: Tela Real de Preferências e Limites dos Sensores

2.24.2. Tabela Detalhada das Telas e Funcionalidades

Nome da Tela	Arquivo	Objetivo	Funcionalidades	APIs	Contextos	Hooks	Componentes	Banco Local
Login	LoginScreen.tsx	Acesso do usuário	Validação, login, link cadastro	POST /auth/login	AuthContext	useLogin, useAuth	View, Text, etc	AsyncStorage
Cadastro	RegisterScreen.tsx	Registro de contas	Criação de novos usuários	POST /auth/registro	AuthContext	useAuth	View, Text, etc	AsyncStorage
Dashboard	DashboardScreen.tsx	Monitoramento	Sensores, polling, gráfico	GET /dispositivos, etc	AuthContext, ThemeContext	useNavigation, etc	ScrollView, LineChart	SQLite/AsyncStorage
Alertas	AlertasScreen.tsx	Histórico	Filtros, cálculo alarmes	GET /leituras, etc	ThemeContext	useTheme, etc	SectionList, etc	SQLite/AsyncStorage
Meus Robôs	MeusRobosScreen.tsx	Gerenciar dispositivos	Listagem, busca, remover	GET /dispositivos, etc	ThemeContext	useNavigation, etc	FlatList, etc	SQLite/AsyncStorage
Novo Robô	NovoRoboScreen.tsx	Parear novo berço	Formulário pareamento	POST /dispositivos	ThemeContext	useNavigation, etc	View, Text, etc	SQLite/AsyncStorage
Renomear	RenomearRoboScreen.tsx	Editar nome	Alterar apelido, sincronizar	PATCH /dispositivos/:uuid	ThemeContext	useNavigation, etc	View, Text, etc	SQLite/AsyncStorage
Robô Detalhes	RoboDetalhesScreen.tsx	Visualizar dados	Status online, botões	GET /dispositivos, etc	ThemeContext	useNavigation, etc	View, Text, etc	SQLite/AsyncStorage
Perfil	PerfilScreen.tsx	Editar conta	Nome, Telefone, Foto	PATCH /usuarios/:id	AuthContext, ThemeContext	useTheme, etc	ScrollView, Image	AsyncStorage
Configurações	ConfiguracoesScreen.tsx	Preferências	Standby, tema, limites	GET /external/api, etc	AuthContext, ThemeContext	useAuth, etc	ScrollView, Switch	SQLite/AsyncStorage

26. Fluxos do Usuário

- **Pareamento:** Usuário abre a aba "Robôs" -> Clica em "Configurar agora" -> Insere o UUID da etiqueta e um nome -> Clica em "Conectar Robô". O app envia a requisição HTTP POST para a API do backend, vinculando a conta do usuário logado ao ID do hardware. O registro é criado localmente no SQLite.
- **Leitura:** Usuário visualiza a aba "Início". O app lê o standby no Blynk. Caso o standby esteja inativo (aparelho ligado), ele realiza buscas periódicas das leituras ambientais e as exibe nos cartões com animação de barras e atualiza o gráfico. As leituras novas são guardadas no SQLite local.
- **Offline:** O usuário perde a internet. O app tenta ler as leituras ou eventos via API, falha no timeout, captura o erro e lê a tabela local `leituras` e `eventos` do SQLite, renderizando os dados agregados na interface.

27. Endpoints da API

2.26.1. Grupo: Autenticação (`/auth`)

70. `POST /auth/registro`

- ***Controller*:** `authController.ts (registro)`
- ***Autenticação*:** Nenhuma (Pública).
- ***Payload*:** `{ "nome": "String", "email": "String", "password": "String", "telefone": "String | null" }`
- ***Resposta (201)*:** `{ "sucesso": true, "mensagem": "...", "dados": { "token": "JWT", "usuario": { ... } } }`
- ***Resposta (400)*:** `{ "sucesso": false, "erros": ["Mensagem de validação"] }`
- ***Resposta (409)*:** `{ "sucesso": false, "mensagem": "Email já cadastrado" }`

71. `POST /auth/login`

- ***Controller*:** `authController.ts (login)`
- ***Autenticação*:** Nenhuma (Pública).

- ***Payload*:** { "email": "String", "password": "String" }
- ***Resposta (200)*:** { "sucesso": true, "dados": { "token": "JWT", "usuario": { ... }, "dispositivos": [...] } }
- ***Resposta (401)*:** { "sucesso": false, "mensagem": "Email ou senha inválidos" }

72. **POST /auth/logout**

- ***Controller*:** authController.ts (logout)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Payload*:** Nenhum.
- ***Resposta (200)*:** { "sucesso": true, "mensagem": "Logout realizado com sucesso" }

73. **GET /auth/validar**

- ***Controller*:** authController.ts (validarToken)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": { "token_valido": true, "usuario": { ... } } }

2.26.2. Grupo: Blynk (/blynk)

74. **GET /blynk/teste/:token**

- ***Controller*:** blynkController.ts (testarBlynk)
- ***Autenticação*:** Nenhuma (Pública).
- ***Resposta (200)*:** { "sucesso": true, "dados": { "v3": "...", "v4": "..." } }
- ***Resposta (502)*:** { "sucesso": false, "mensagem": "Falha ao comunicar com o Blynk..." }

2.26.3. Grupo: Usuários (/usuarios)

75. **GET /usuarios**

- ***Controller*:** usuariosController.ts (listarUsuarios)
- ***Autenticação*:** Obrigatória (Bearer JWT).

- ***Query Params*:** page (default 1), limit (default 10).
- ***Resposta (200)*:** { "sucesso": true, "dados": { "pagina", "limite", "quantidade", "usuarios": [...] } }

76. **GET /usuarios/:id**

- ***Controller*:** usuariosController.ts (buscarUsuarioPorId)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": { "id_usuario", "nome", ... } }

77. **POST /usuarios**

- ***Controller*:** usuariosController.ts (criarUsuario)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Payload*:** { "nome", "email", "senha", "telefone?" }
- ***Resposta (201)*:** { "sucesso": true, "dados": { ... } }

78. **PATCH /usuarios/:id**

- ***Controller*:** usuariosController.ts (atualizarUsuario)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Payload*:** { "nome?", "email?", "senha?", "telefone?", "foto_perfil?", "push_token?" }
- ***Resposta (200)*:** { "sucesso": true, "dados": { ... } }

79. **DELETE /usuarios/:id**

- ***Controller*:** usuariosController.ts (deletarUsuario)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "mensagem": "Usuário deletado com sucesso" }

2.26.4. Grupo: Dispositivos (/dispositivos)

80. **GET /dispositivos**

- ***Controller*:** dispositivosController.ts (listarDispositivos)

- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": [...] }

81. **POST /dispositivos**

- ***Controller*:** dispositivosController.ts (criarDispositivo)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Payload*:** { "nome_dispositivo": "Ex", "uuid_dispositivo": "Ex" }
- ***Resposta (201)*:** { "sucesso": true, "dados": { ... } }
- ***Resposta (404)*:** { "sucesso": false, "mensagem": "Código do dispositivo inválido ou não reconhecido pela fábrica." }

82. **PATCH /dispositivos/:uuid**

- ***Controller*:** dispositivosController.ts (atualizarDispositivo)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Payload*:** { "nome_dispositivo?", "status_dispositivo?" }
- ***Resposta (200)*:** { "sucesso": true, "dados": { ... } }

83. **DELETE /dispositivos/:uuid**

- ***Controller*:** dispositivosController.ts (deletarDispositivo)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "mensagem": "Dispositivo removido com sucesso do banco de dados" }

2.26.5. Grupo: Sensores (/sensores)

84. **GET /sensores**

- ***Controller*:** sensoresController.ts (listarSensores)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": { "quantidade", "sensores" } }

85. **GET /sensores/:id**

- ***Controller*:** sensoresController.ts (buscarSensorPorId)

- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": { ... } }

86. **POST /sensores**

- ***Controller*:** sensoresController.ts (criarSensor)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Payload*:** { "id_dispositivo", "nome_sensor?", "tipo_sensor", "unidade_medida?", "descricao?" }
- ***Resposta (210)*:** { "sucesso": true, "dados": { ... } }

87. **DELETE /sensores/:id**

- ***Controller*:** sensoresController.ts (deletarSensor)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "mensagem": "Sensor removido com sucesso" }

2.26.6. Grupo: Leituras (/leituras)

88. **GET /leituras**

- ***Controller*:** leiturasController.ts (listarLeituras)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": { "quantidade", "leituras" } } (Contém histórico agrupado por minuto).

89. **GET /leituras/dispositivo/:id_dispositivo**

- ***Controller*:** leiturasController.ts (buscarUltimasLeiturasPorDispositivo)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": { "quantidade", "leituras" } } (Retorna a última leitura de cada sensor).

90. **GET /leituras/:id**

- ***Controller*:** leiturasController.ts (buscarLeituraPorId)
- ***Autenticação*:** Obrigatória (Bearer JWT).

- ***Resposta (200)*:** { "sucesso": true, "dados": { ... } }

91. **POST /leituras**

- ***Controller*:** leiturasController.ts (criarLeitura)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Payload*:** { "id_sensor", "valor?", "valor_booleano?", "movimento?" }
- ***Resposta (201)*:** { "sucesso": true, "dados": { ... } }

92. **DELETE /leituras/limpar/todos**

- ***Controller*:** leiturasController.ts (limparLeituras)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "mensagem": "Histórico de leituras limpo com sucesso..." }

93. **DELETE /leituras/:id**

- ***Controller*:** leiturasController.ts (deletarLeitura)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "mensagem": "Leitura removida com sucesso" }

2.26.7. Grupo: Eventos (/eventos)

94. **GET /eventos**

- ***Controller*:** eventosController.ts (listarEventos)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": [...] } (Filtrar eventos apenas dos robôs do usuário logado).

95. **GET /eventos/:id**

- ***Controller*:** eventosController.ts (buscarEventoPorId)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "dados": { ... } }

96. `POST /eventos`

- ***Controller*:** eventosController.ts (criarEvento)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Payload*:** { "id_dispositivo", "id_sensor?", "tipo_evento", "nivel_criticidade" }
- ***Resposta (201)*:** { "sucesso": true, "dados": { ... } }

97. `DELETE /eventos/limpar/todos`

- ***Controller*:** eventosController.ts (limparEventos)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "mensagem": "Histórico limpo com sucesso..." }

98. `DELETE /eventos/:id`

- ***Controller*:** eventosController.ts (deletarEvento)
- ***Autenticação*:** Obrigatória (Bearer JWT).
- ***Resposta (200)*:** { "sucesso": true, "mensagem": "Evento deletado com sucesso" }

28. Regras de Negócio

- **Segurança do Berço (Movimento):** A leitura física da distância do sensor HC-SR04 é convertida em boolean de movimento. Valores inferiores a 50cm caracterizam presença ou agitação no berço e disparam flag de movimento ativa.
- **Ativação e Standby (V7):** O interruptor no app envia o comando de Standby (V7 = 0) para o Blynk Cloud. Ao ler V7 = 0, o backend marca o dispositivo como 'offline' e interrompe a gravação das leituras no banco. O app móvel detecta o standby em tempo real e exibe banners, ocultando o gráfico.
- **Isolamento de Contas:** Os eventos e dispositivos são estritamente filtrados pelo `id_usuario` autenticado. Um usuário não tem autorização ou visibilidade para dados de hardware de terceiros.

- **Pareamento de fábrica (Claim Device):** Um dispositivo só pode ser pareado pelo aplicativo se seu UUID já existir no PostgreSQL. O pareamento vincula o `id_usuario` logado à chave física. Se outra conta tentar parear um UUID já associado, retorna erro de conflito (409).
- **Trava de Controle do Standby:** Caso o usuário logado não possua nenhum robô (dispositivo) associado à sua conta no banco, o switch de Standby na aba Ajustes é desabilitado/bloqueado e renderiza em vermelho a instrução: 'Cadastre um dispositivo para utilizar esta função.'
- **Comportamento em Standby (Dashboard):** Quando o standby físico está ativo ($V7 = 0$), a Dashboard zera a exibição dos sensores com traços (—), oculta o gráfico de variação de temperatura substituindo-o por um aviso informativo e exibe um banner principal indicando o 'Modo Standby' (notificações e monitoramento pausados).
- **Atualização em Foco:** A tela de configurações utiliza o hook `useFocusEffect` para atualizar instantaneamente as informações do dispositivo ativo e do estado Blynk sempre que a tela ganha foco.

29. Segurança Implementada

- **Tokens JWT:** Criptografados com assinatura HMAC-SHA256, expirando periodicamente.
- **Blacklist local de logout:** Armazenamento em banco (`tokens_revogados`) para revogação imediata.
- **Hashing de Senhas:** Uso de saltos com `bcryptjs` de complexidade 10.
- **SQL Injection Prevention:** Uso estrito de queries parametrizadas (ex: `$1`, `$2` no Postgres, e `?` no SQLite).
- **Sanitização:** Strings tratadas contra scripts e injeções antes da persistência.

30. Tratamento de Erros

- **Backend:** Middlewares interceptam exceções e retornam respostas estruturadas com código HTTP correspondente (400, 401, 403, 404, 409, 500) e mensagem amigável no formato JSON.

- **Frontend:** Uso de blocos `try/catch` envolvendo chamadas a recursos nativos (imagem/câmera) e requisições externas para prevenir travamento completo (crash) do app móvel. Caso ocorra erro de rede, o app faz o fallback offline exibindo logs no console e Toast de alerta ao usuário.

31. Boas Práticas Aplicadas

- **Clean Code:** Arquitetura modular separando responsabilidades (MVC / MVVM / Repository).
- **Organização das Pastas:** Estrutura organizada e clara no frontend e backend.
- **Código limpo e tipado:** Uso rígido de TypeScript, com tipos e interfaces explícitos para contratos de APIs e schemas de tabelas.
- **Otimização de Renderização:** Uso de `useCallback` e `useMemo` no `SectionList` de alertas para diminuir o custo de reinicialização e evitar re-renders na listagem.

32. Processo de Instalação

2.31.1. Backend

99. Navegue até a pasta `backend/`.

100. Execute o comando para instalar as dependências:

```
npm install
```

2.31.2. Frontend

101. Navegue até a pasta `frontend/`.

102. Execute o comando para instalar as dependências:

```
npm install
```

33. Processo de Execução

2.32.1. Inicializando o Banco de Dados PostgreSQL

Execute as tabelas remotas no Supabase/PGAdmin conforme a seção 13 (PostgreSQL).

2.32.2. Configuração das Variáveis de Ambiente (`.env`)

No diretório `backend/.env`:

```
DATABASE_URL=postgresql://usuario:senha@host:porta/banco?sslmode=require
JWT_SECRET=sua_chave_secreta_jwt
```

No diretório `frontend/.env`:

```
EXPO_PUBLIC_API_URL=http://<IP_DO_SEU_COMPUTADOR>:3000
EXPO_PUBLIC_BLYNK_TOKEN=seu_blynk_auth_token
EXPO_PUBLIC_BLYNK_BASE_URL=https://blynk.cloud/external/api
```

2.32.3. Executando o Servidor de Backend

No diretório `backend/`:

```
npm run dev
```

2.32.4. Executando o Aplicativo Móvel (Expo)

No diretório `frontend/`:

```
npx expo start
```

- Use a tecla `a` para abrir no emulador Android, ou escaneie o QR Code no aparelho físico via Expo Go.

34. Ambiente de Desenvolvimento

- **OS:** Windows / macOS / Linux
- **Node.js:** v18 ou superior.
- **Expo Go:** v54 instalado no celular para validação local.
- **Wokwi:** Para simulação física da placa ESP32 enviando telemetria.

35. Limitações Conhecidas

- **Dependência Blynk:** Se os servidores do Blynk Cloud ficarem lentos, o tempo de resposta do dashboard do aplicativo móvel ou do worker do backend será afetado.

- **Limites de Polling:** O polling de 10 segundos do backend impede o monitoramento em tempo real instantâneo (sub-segundo) na base de dados de histórico remoto, de modo que variações rápidas só aparecem na tela do Blynk, mas não no histórico do aplicativo.
- **Fallback Local Estático:** O SQLite local do aplicativo móvel só recebe cache ao abrir e ler as telas sob conexão de internet. Caso o app seja aberto pela primeira vez sem nenhuma conexão prévia, o SQLite estará vazio.

36. Conclusão

O BabyGuard atende a todos os critérios e requisitos exigidos no regulamento da 2ª etapa de PPDM. A integração de persistência dupla (AsyncStorage para chave-valor rápida e SQLite para cache estruturado local) garante que o aplicativo funcione perfeitamente de forma offline, e a modularização baseada em clean code simplifica manutenções futuras.